



Actividad

Examen: Diseño de sistemas y Base de Datos

Nombre de la escuela

Universidad Politécnica de Chiapas

Autores

243763 Rudi Fabricio Martínez Jaimes
243726 Gonzalez Ruiz Andres Esuardo
243743 Mora Mercado Fernando

Docente

Ramsés Camas Nájera

Fecha

05/02/2026

Lugar

Suchiapa, Chiapas, México

Materia

BASES DE DATOS AVANZADAS

Grado y Grupo

5 B

Informe de Implementación: Sistema de Gestión Logística Regional (Caso 3-A)

1. Introducción

El presente documento detalla el diseño y desarrollo de una solución de base de datos para una distribuidora logística en Tuxtla Gutiérrez. El sistema ha sido diseñado para gestionar la cadena de suministro integral: desde el reabastecimiento por proveedores hasta la entrega final con incidencias en las zonas de Chiapas, Oaxaca y Tabasco. El enfoque principal fue garantizar la integridad referencial, la eliminación de redundancias y la resiliencia ante errores de lógica de negocio.

2. Desarrollo del Proyecto Paso a Paso

Fase 1: Análisis y Modelado Conceptual

Se realizó un mapeo de entidades basado en el escenario real. Para cumplir con la complejidad alta requerida, el modelo se normalizó a Tercera Forma Normal (3FN), estructurando 12 tablas interconectadas.

- **Identificación de Entidades:** Se separaron los catálogos maestros (Proveedores, Zonas, Categorías) de las tablas transaccionales (Entradas, Pedidos, Rutas).
- **Decisión Estratégica:** Se optó por un modelo donde el Comercio está ligado a una Zona, permitiendo análisis geográficos sin duplicidad de datos.

Fase 2: Arquitectura del Esquema y Relaciones Complejas

Durante la implementación del schema.sql, se resolvieron los desafíos técnicos de cardinalidad:

1. **Relaciones Muchos a Muchos (\$N:N\$):** * **Venta:** Detalle_Pedido vincula Productos con Pedidos, permitiendo trazabilidad individual por artículo.
 - **Logística:** Carga_Ruta vincula Rutas con Pedidos, permitiendo que una ruta consolide múltiples entregas y que un pedido pueda ser gestionado logísticamente de forma independiente.
2. **Entidad Débil:** Se implementó Entradas_Almacen, la cual depende de la existencia de un Producto y un Proveedor, registrando el flujo de stock inicial (ej. los 500L de leche).
3. **Capa de Integridad (Constraints):** Se programaron 10 restricciones clave, incluyendo:
 - CHECK para evitar stocks negativos y asegurar que el precio_venta > precio_compra.
 - UNIQUE en campos de identidad (RFC, Licencia).
 - NOT NULL en claves foráneas para evitar registros "huérfanos".

Fase 3: Escenarios de Prueba (Seeding)

La validación del modelo se realizó mediante el script seeds.sql, recreando la narrativa del caso:

- **Gestión de Activos:** Se registró la camioneta AB-123-CD y al conductor Carlos Hernández.
- **Flujo de Incidencias:** Se simuló la entrega en Juchitán, donde la relación \$N:N\$ permitió marcar específicamente 20kg de queso como dañados en la tabla de incidencias, sin afectar la integridad del resto de la carga.

Fase 4: Inteligencia de Negocio (Views)

Se desarrollaron 5 vistas (VIEWS) que transforman datos crudos en decisiones:

- **Inventario Crítico:** Utiliza COALESCE y NULLIF para prevenir errores de división por cero en productos sin venta.
- **Productividad y Flota:** Clasifican el desempeño mediante sentencias CASE, asignando rangos de rendimiento de forma automática.

3. Justificación de Decisiones Técnicas

¿Por qué escoger el Caso 3-A?

Elegimos la Logística de Distribución Regional por su alta demanda de lógica relacional. A diferencia de un sistema de ventas simple, este caso requiere coordinar tres dimensiones en tiempo real: Inventario (entradas/salidas), Comercial (pedidos/clientes) y Operativa (vehículos/rutas/incidencias). Representa un reto de ingeniería de datos donde un error en una cardinalidad puede detener la operación de una empresa real.

¿Por qué diseñarlo así?

Principalmente por tres aspectos importantes:

1. **Desacoplamiento Operativo:** Al separar la Ruta del Pedido mediante una tabla puente, el sistema es escalable. Si un pedido excede la capacidad de un vehículo, el modelo permite dividirlo en dos rutas distintas sin alterar el folio del pedido original.
2. **Preservación de Históricos:** En la tabla Detalle_Pedido capturamos el precio al momento de la venta. Esto evita que cambios futuros en el catálogo de productos alteren retroactivamente los reportes financieros de meses anteriores.
3. **Gestión Atómica de Incidencias:** Al crear una tabla específica para incidencias vinculada al detalle del producto, permitimos que el negocio identifique patrones. No solo sabemos que hubo un problema en Juchitán; sabemos que el problema es recurrentemente con el queso de un proveedor específico, permitiendo una toma de decisiones basada en datos.

Views

```
-- 1. Resumen de pedidos por zona geográfica
CREATE VIEW vista_resumen_zonas AS
SELECT
  z.nombre AS zona,
  COUNT(p.id_pedido) AS total_pedidos,
  SUM(CASE WHEN p.estado_pedido = 'Entregado' THEN 1 ELSE 0 END) AS exitosos,
  SUM(CASE WHEN p.estado_pedido = 'Entregado con incidencia' THEN 1 ELSE 0 END) AS con_incidencia,
  ROUND(CAST(SUM(CASE WHEN p.estado_pedido = 'Entregado' THEN 1 ELSE 0 END) AS NUMERIC) / NULLIF(COUNT(p.id_pedido), 0) * 100, 2) || '%' AS tasa_exito
FROM Zonas z
JOIN Comercios c ON z.id_zona = c.id_zona
JOIN Pedidos p ON c.id_comercio = p.id_comercio
GROUP BY z.nombre
HAVING COUNT(p.id_pedido) > 0;
```

```
-- 2. Inventario Crítico
CREATE VIEW vista_inventario_critico AS
SELECT
  p.nombre,
  p.stock_actual,
  COALESCE(SUM(dp.cantidad), 0) AS ventas_totales_periodo,
  CASE
    WHEN COALESCE(SUM(dp.cantidad), 0) = 0 THEN 'Sin ventas recientes - Validar baja rotación'
    WHEN p.stock_actual / (CAST(SUM(dp.cantidad) AS NUMERIC) / 30.0) < 15 THEN 'CRÍTICO: Stock para < 15 días'
    ELSE 'Stock Suficiente'
  END AS estatus_alerta
FROM Productos p
LEFT JOIN Detalle_Pedido dp ON p.id_producto = dp.id_producto
GROUP BY p.id_producto, p.nombre, p.stock_actual;
```

```

-- 3. Productividad de Conductores (Actualizado con Ruta_Pedidos)
CREATE VIEW vista_productividad_conductores AS
SELECT
    c.nombre,
    COUNT(DISTINCT r.id_ruta) AS rutas_totales,
    COALESCE(COUNT(rp.id_pedido), 0) AS total_pedidos_entregados,
    COALESCE(AVG(p_stats.peso_ruta), 0) AS promedio_carga_por_ruta,
    CASE
        WHEN COUNT(DISTINCT r.id_ruta) > 20 THEN 'Alto Rendimiento'
        WHEN COUNT(DISTINCT r.id_ruta) BETWEEN 5 AND 20 THEN 'Estándar'
        ELSE 'En desarrollo / Baja actividad'
    END AS clasificacion_rendimiento
FROM Conductores c
LEFT JOIN Rutas r ON c.id_conductor = r.id_conductor
LEFT JOIN Ruta_Pedidos rp ON r.id_ruta = rp.id_ruta
LEFT JOIN (
    SELECT
        rp.id_ruta,
        SUM(p.total_peso) as peso_ruta
    FROM Ruta_Pedidos rp
    JOIN Pedidos p ON rp.id_pedido = p.id_pedido
    GROUP BY rp.id_ruta
) p_stats ON r.id_ruta = p_stats.id_ruta
GROUP BY c.id_conductor, c.nombre;

```

```

-- 4. Rentabilidad por proveedor (Usa precio historico)
CREATE VIEW vista_rentabilidad_proveedor AS
SELECT
    pr.nombre AS proveedor,
    SUM(dp.cantidad * p.precio_compra) AS costo_ventas,
    SUM(dp.cantidad * dp.precio_venta_historico) AS ventas_totales, -- Usando precio historico
    SUM(dp.cantidad * (dp.precio_venta_historico - p.precio_compra)) AS margen_bruto_ganancia,
    ROUND((SUM(dp.cantidad * (dp.precio_venta_historico - p.precio_compra)) / NULLIF(SUM(dp.cantidad * dp.precio_venta_historico), 0)) * 100, 2) || '%' as margen_porcentual
FROM Proveedores pr
JOIN Productos p ON pr.id_proveedor = p.id_proveedor
JOIN Detalle_Pedido dp ON p.id_producto = dp.id_producto
GROUP BY pr.id_proveedor, pr.nombre;

```

```
-- 5. Utilización de flota
CREATE VIEW vista_utilizacion_flota AS
SELECT
    v.placas,
    v.modelo,
    COUNT(r.id_ruta) AS rutas_en_mes,
    CASE
        WHEN COUNT(r.id_ruta) = 0 THEN 'Subutilizado / Parado'
        WHEN COUNT(r.id_ruta) > 25 THEN 'Sobrecargado'
        ELSE 'Operación Óptima'
    END AS diagnostico_uso
FROM Vehiculos v
LEFT JOIN Rutas r ON v.id_vehiculo = r.id_vehiculo
GROUP BY v.id_vehiculo, v.placas, v.modelo;
```